PIMPRI CHINCHWAD EDUCATION TRUST'S

Pimpri Chinchwad College of Engineering (PCCoE)

(An Autonomous Institute) — Affiliated to Savitribai Phule Pune University (SPPU)

Vehicle Service Center Management System

FA1 — Design to Database | Object Oriented Modelling and System Design [BCER27PC02]

GROUP MEMBERS

Anushka Misal	123B1D027
Yadnesh Patil	123B1D034
Abhay Bormale	123B1D057

Case Study / Problem Statement

Today: A Manual, Paper-Based Workflow

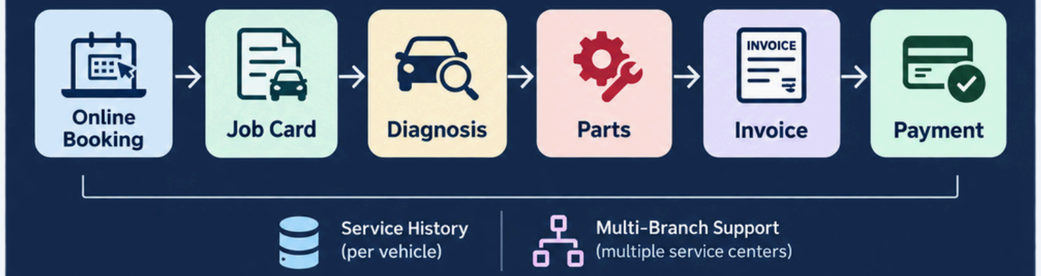
- Customers book service by walk-in or phone call
- Advisors manually note vehicle problems on paper job cards
- Mechanics are assigned verbally — no formal record
- Spare parts are picked from shelves with no stock tracking
- Bills are handwritten, invoices are error-prone

Resulting Problems

- Appointment clashes
- Lost job cards
- Incorrect billing
- No visibility into service history or parts inventory

The Proposed System

Automates the workflow end-to-end: online booking → job card → diagnosis → parts → invoice → payment, with full service history and multi-branch support.



SCOPE HIGHLIGHTS

- Customer & vehicle registration with slot-availability booking
- Job-card based tracking of diagnosis, repairs & labor
- Real-time spare-parts inventory, auto-deducted on use
- Automatic invoicing (labor + parts + tax) & payment recording
- Multi-branch support with dedicated advisors & mechanics
- Service history per vehicle + admin revenue/inventory reports

CRC Cards — Class, Responsibility, Collaborator

Core classes identified from the customer-facing and booking workflow

CLASS	RESPONSIBILITIES	COLLABORATORS
Customer	Register/login; maintain personal details; register vehicles; book appointments; view service history; make payments.	Vehicle, Appointment, Invoice, Payment
Vehicle	Store registration number, make, model and year; provide vehicle details for service.	Customer, Appointment, JobCard
Appointment	Check slot availability; create and confirm appointments; cancel bookings; link customer, vehicle and service center.	Customer, Vehicle, ServiceCenter, JobCard
ServiceCenter	Maintain branch information; manage branch advisors and mechanics.	Appointment, ServiceAdvisor, Mechanic
ServiceAdvisor	Create job cards from confirmed appointments; assign mechanics; generate invoices; notify customers.	Appointment, JobCard, Mechanic, Invoice
Mechanic	Diagnose vehicles; perform repair work; update job status; record spare parts used.	JobCard, SparePart, Vehicle

CRC Cards — Continued

Core classes for job execution, inventory and billing

CLASS	RESPONSIBILITIES	COLLABORATORS
JobCard	Track diagnosis and work performed; store dates and labor cost; maintain parts used; support invoice generation.	Appointment, ServiceAdvisor, Mechanic, JobCardPart, Invoice
SparePart	Maintain part name, unit price and stock quantity; check availability; update stock.	JobCardPart
JobCardPart	Link a job card with a spare part; record quantity used for each part.	JobCard, SparePart
Invoice	Calculate labor + parts + tax; calculate total amount; maintain invoice status.	JobCard, Payment
Payment	Record payment amount, mode, date and status for an invoice.	Invoice, Customer

GRASP & SOLID Principles Applied

1

Information Expert

JobCard handles job-related labor information, while SparePart handles stock and price information.

2

Creator

ServiceAdvisor creates a JobCard from a confirmed appointment; Customer creates an Appointment.

3

Controller

Appointment coordinates appointment-related actions such as checking availability and confirming or cancelling a booking.

4

Low Coupling

JobCard and SparePart are connected through JobCardPart, which handles the many-to-many relationship.

5

High Cohesion

Each class has a focused responsibility, such as Invoice for billing and Payment for payment records.

6

Single Responsibility

Invoice handles billing calculations while Payment handles payment records.

7

Open/Closed Principle

New service-related behaviour can be added without changing the existing invoice calculation.

UML Class Diagram

The class diagram reflects the classes identified in the CRC cards and shows their attributes, methods, relationships and multiplicities.

JobCard sits at the center, linking Appointment, ServiceAdvisor, Mechanic and Invoice, with JobCardPart resolving the many-to-many relationship with SparePart.

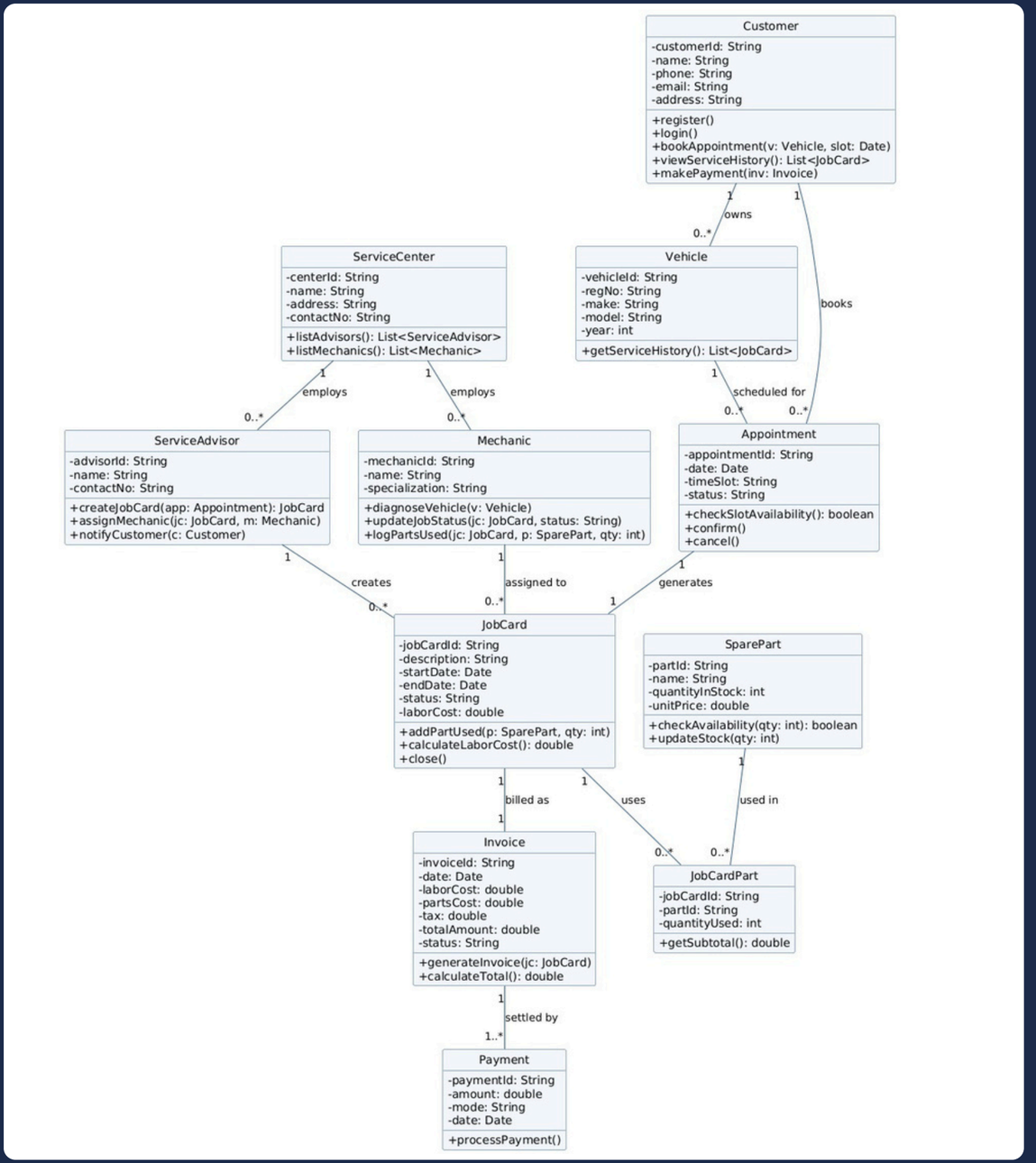


Fig. 1 — UML Class Diagram of the Vehicle Service Center Management System

Use Case Diagram

Four actors interact with the system:
• Customer
• Service Advisor
• Mechanic
• Admin

The diagram shows the main functions and the required include relationships between use cases, such as Check Slot Availability within Book Service Appointment.

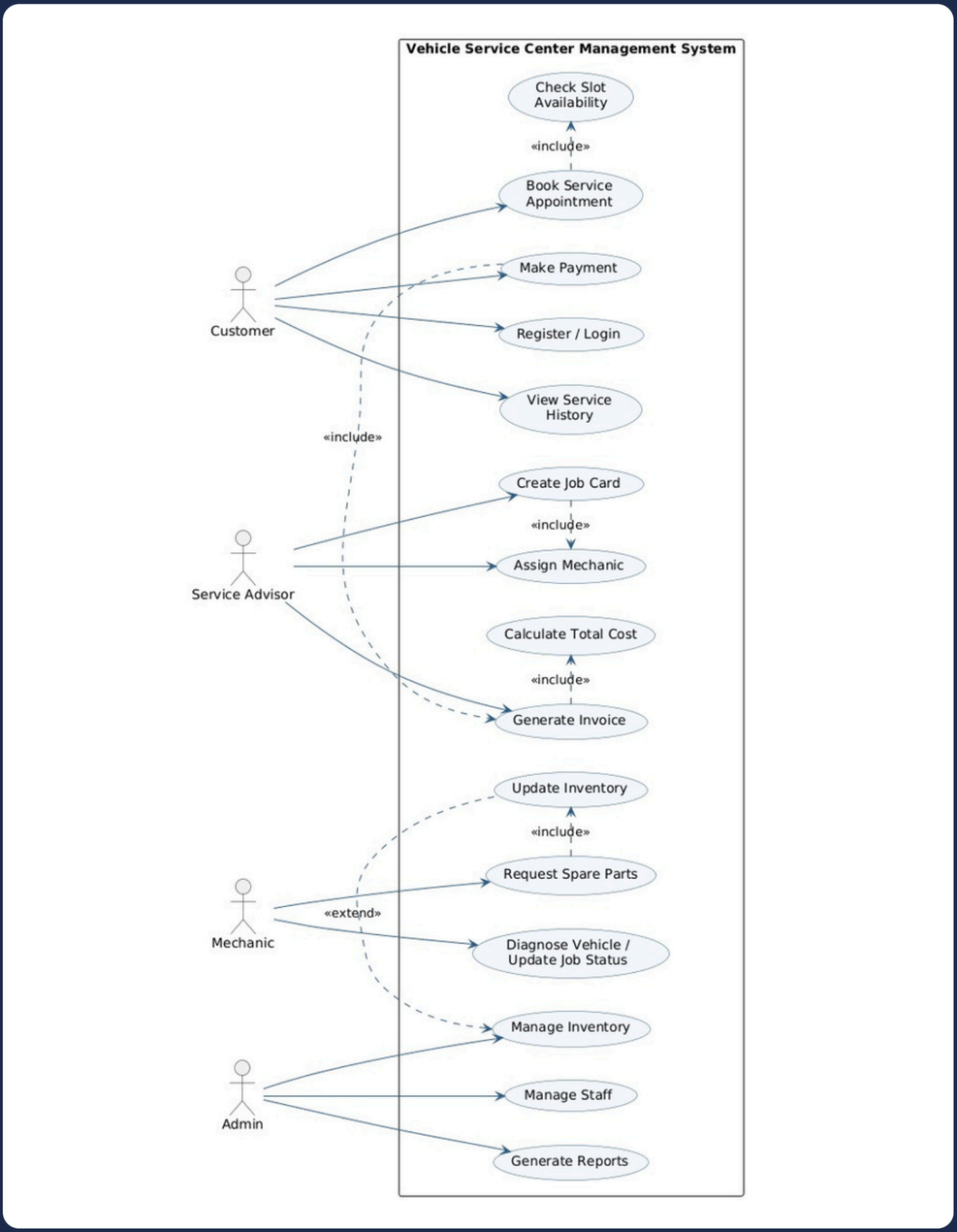


Fig. 2 — Use Case Diagram

Use Case Descriptions

USE CASE	ACTOR(S)	DESCRIPTION
Register / Login	**Customer**	Create an account or authenticate to access the system.
Register Vehicle	**Customer**	Add a vehicle using its registration number and basic details.
Book Service Appointment	**Customer**	Select a vehicle and an available time slot to book a service.
Create Job Card	**Service Advisor**	Convert a confirmed appointment into an active job card.
Diagnose / Update Job Status	**Mechanic**	Inspect the vehicle, perform repairs and update progress.
Generate Invoice	**Service Advisor**	Produce the bill once a job card is completed.
Make Payment	**Customer**	Pay the outstanding invoice amount.
Manage Inventory / Staff	**Admin**	Add/update spare parts, stock levels, advisors and mechanics.
Generate Reports	**Admin**	View revenue, job and inventory reports.

Sequence Diagram — Book Service Appointment

Traces the Book Service Appointment use case through the objects also present in the Class Diagram:

- Customer
- Vehicle
- Appointment
- ServiceCenter
- ServiceAdvisor

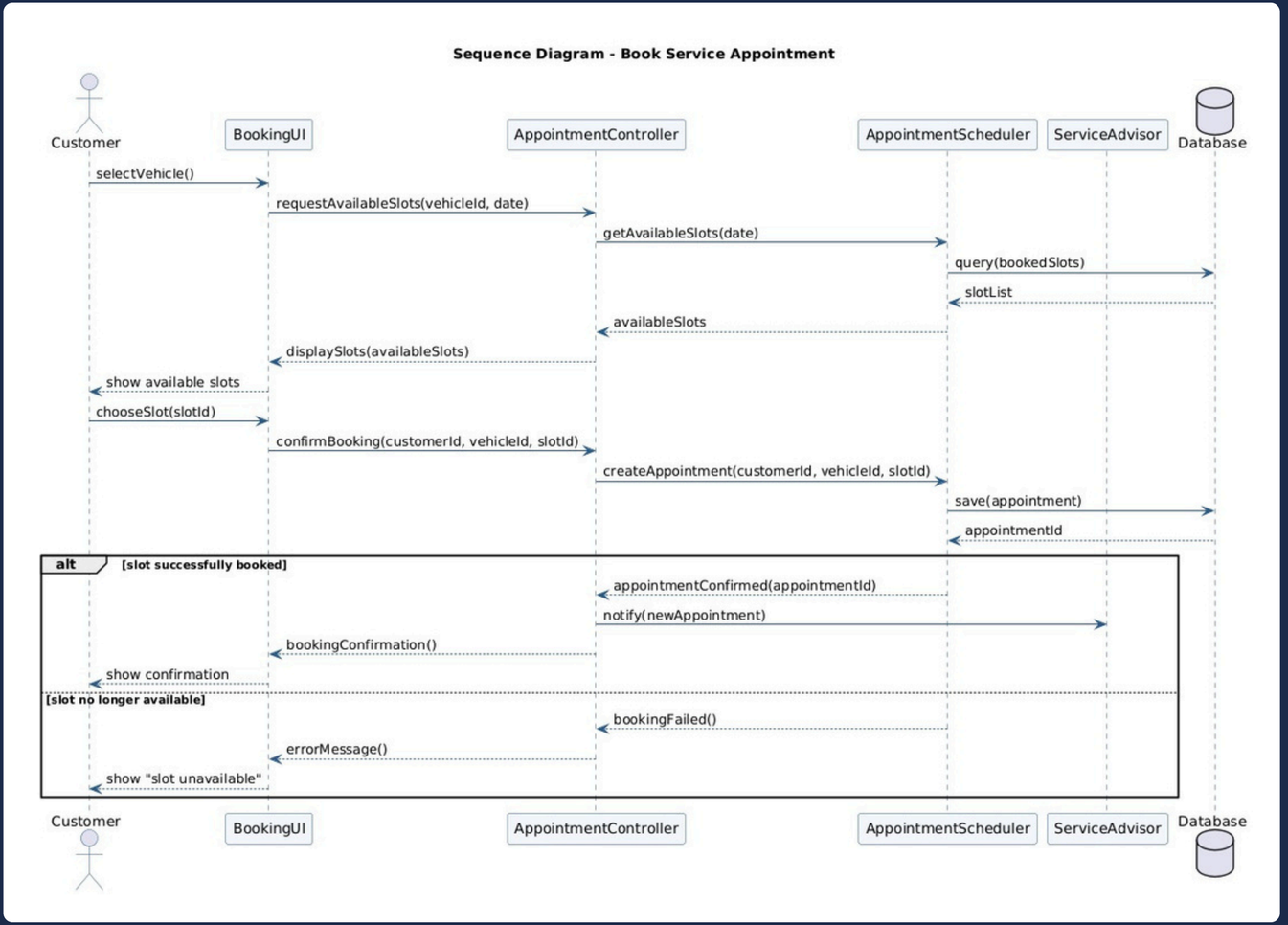


Fig. 3 — Sequence Diagram for Book Service Appointment

Entity-Relationship Diagram

Identifies the entities, attributes, primary keys and foreign keys required for the system.

JOB_CARD_PART is used as the associative entity between JOB_CARD and SPARE_PART, resolving their many-to-many relationship.

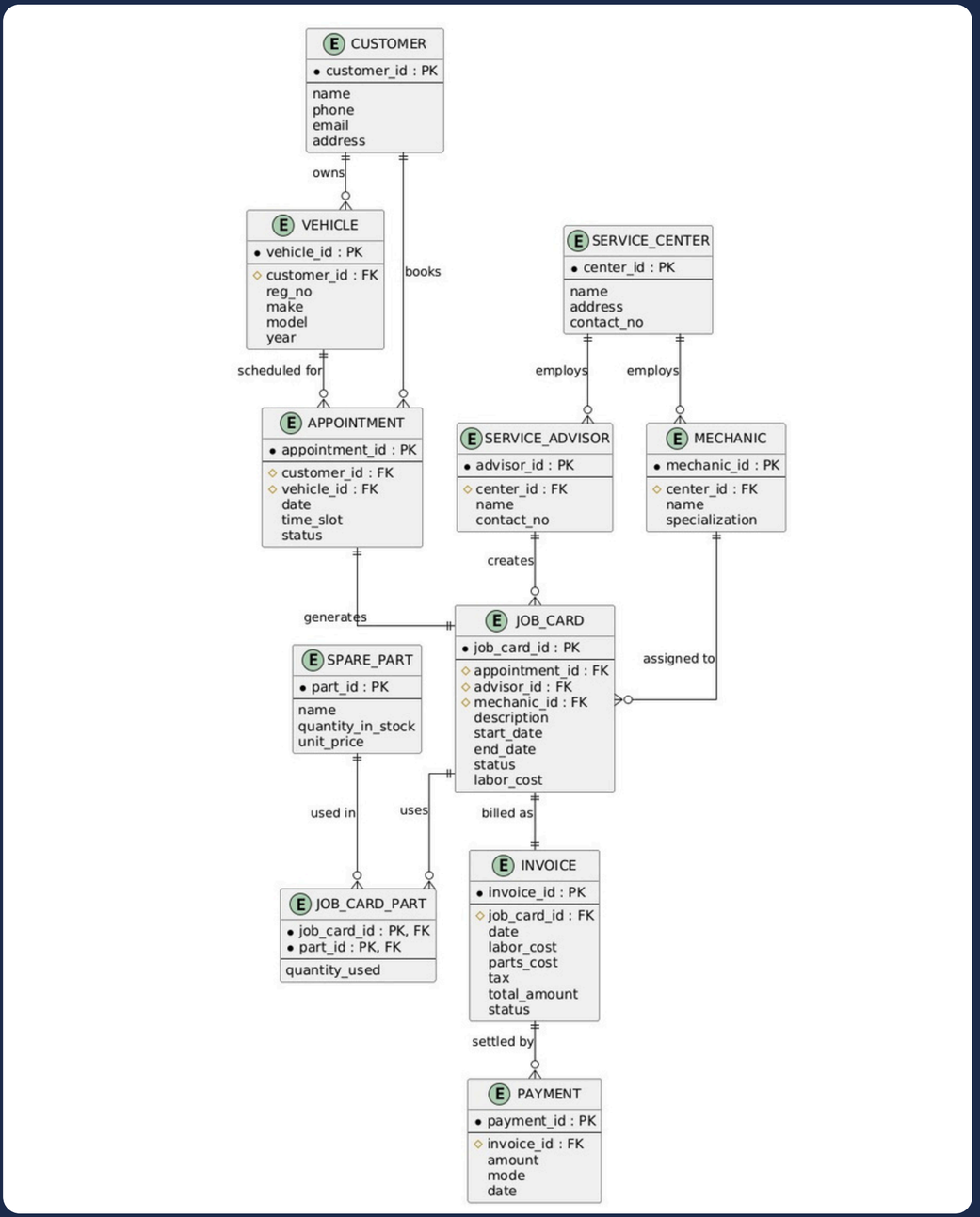


Fig. 4 — Entity-Relationship Diagram

Normalization — Up to 3NF

A traditional flat job-card/bill sheet repeats part information per visit — normalization removes this redundancy.

UNF → 1NF → 2NF → 3NF

UNF — Un-normalized Form

The Parts field contains a repeating group (e.g. “Brake Pad (1,850); Engine Oil (2,450)”), so the relation is not in 1NF.

JobCardID	CustomerName	VehicleRegNo	Parts (Name, Qty, Price)	TotalAmount
JC101	Rahul Shah	MH12AB1234	Brake Pad(1,850); Engine Oil(2,450)	3550
JC102	Neha Joshi	MH14CD5678	Air Filter(1,300)	700

1NF — First Normal Form

Repeating part groups are removed by storing one row per (JobCardID, PartID); values become atomic, with composite key (JobCardID, PartID).

JobCardID	CustomerName	PartID	PartName	Qty	UnitPrice
JC101	Rahul Shah	P01	Brake Pad	1	850
JC101	Rahul Shah	P02	Engine Oil	2	450
JC102	Neha Joshi	P03	Air Filter	1	300

Normalization — 2NF & 3NF

2 Second Normal Form (2NF)

PartName and UnitPrice depend only on PartID; customer, vehicle, advisor and mechanic details depend only on JobCardID. These partial dependencies are removed:

- `JOB_CARD (JobCardID, CustomerID, VehicleID, AdvisorID, MechanicID, LaborCost)`
- `JOB_CARD_PART (JobCardID, PartID, QuantityUsed)`
- `SPARE_PART (PartID, PartName, UnitPrice, QuantityInStock)`

3 Third Normal Form (3NF)

In JOB_CARD, CustomerID, VehicleID, AdvisorID and MechanicID identify details stored in their own tables. Separating these removes transitive dependencies, so every table reaches 3NF.

Final 3NF Schema

Table	Key Attribute(s)	Non-key Attributes
CUSTOMER	customer_id (PK)	name, phone, email, address
VEHICLE	vehicle_id (PK), customer_id (FK)	reg_no, make, model, year
SERVICE_CENTER	center_id (PK)	name, address, contact_no
SERVICE_ADVISOR	advisor_id (PK), center_id (FK)	name, contact_no
MECHANIC	mechanic_id (PK), center_id (FK)	name, specialization
APPOINTMENT	appointment_id (PK), customer_id/vehicle_id/center_id (FK)	date, time_slot, status
JOB_CARD	job_card_id (PK), appointment_id/advisor_id/mechanic_id (FK)	description, start_date, end_date, status, labor_cost
SPARE_PART	part_id (PK)	name, quantity_in_stock, unit_price
JOB_CARD_PART	job_card_id, part_id (PK, FK)	quantity_used
INVOICE	invoice_id (PK), job_card_id (FK)	date, labor_cost, parts_cost, tax, total_amount, status
PAYMENT	payment_id (PK), invoice_id (FK)	amount, mode, date, status